

PHẦN 2: QUẢN LÝ TIẾN TRÌNH

1. Tiến Trình
2. Luồng(**thread**)
3. Điều phối CPU
4. Tài nguyên găng và **đồng bộ** tiến trình
5. Bể tắc và xử lý bể tắc

1.3. Thao tác trên tiến trình

- Tạo tiến trình
- Kết thúc tiến trình

Tạo tiến trình

- Tiến trình có thể tạo nhiều tiến trình mới cùng hoạt động (CreateProcess(), fork())
 - Tiến trình tạo: tiến trình cha
 - Tiến trình được tạo: tiến trình con
- Tiến trình con có thể tạo tiến trình con khác -> cây tiến trình
- Vấn đề phân phối tài nguyên
 - Tiến trình con lấy tài nguyên từ hệ điều hành
 - Tiến trình con lấy tài nguyên từ tiến trình cha
 - Tất cả các tài nguyên
 - Một phần tài nguyên của tiến trình cha (ngăn ngừa việc tạo quá nhiều tiến trình con)
- Vấn đề thực hiện
 - Tiến trình cha tiếp tục thực hiện đồng thời với tiến trình con
 - Tiến trình cha đợi tiến trình con kết thúc

Kết thúc tiến trình

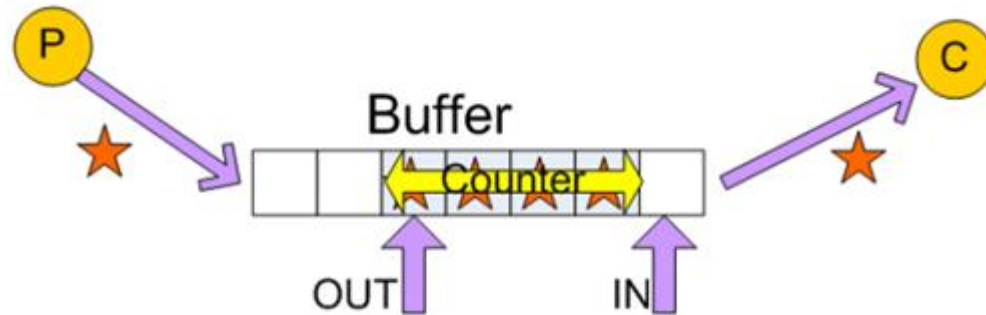
- Hoàn thành câu lệnh cuối và yêu cầu HĐH xóa nó (exit)
 - Gửi trả dữ liệu tới tiến trình cha
 - Các tài nguyên đã cung cấp được trả lại hệ thống
- Tiến trình cha có thể kết thúc sự thực hiện của tiến trình con
 - Tiến trình cha phải biết định danh tiến trình con -> tiến trình con phải gửi định danh cho tiến trình cha khi được khởi tạo
 - Sử dụng lời gọi hệ thống (abort)
- Tiến trình cha kết thúc tiến trình con khi
 - Tiến trình con sử dụng vượt quá tài nguyên được cấp
 - Nhiệm vụ cung cấp cho tiến trình con không còn cần thiết nữa
 - Tiến trình cha kết thúc và hệ điều hành không cho phép tiến trình con tồn tại khi tiến trình cha kết thúc -> Cascading termination . VD, kết thúc hệ thống

1.4. Phân loại tiến trình

- Các tiến trình tuần tự
 - Điểm bắt đầu của tiến trình này nằm sau điểm kết thúc của tiến trình kia
- Các tiến trình song song
 - Điểm bắt đầu của tiến trình này **nằm giữa điểm bắt đầu và kết thúc** của tiến trình kia
 - **Độc lập:** Không ảnh hưởng tới hoặc bị ảnh hưởng bởi tiến trình khác đang thực hiện trong hệ thống
 - **Có hợp tác:** Ảnh hưởng tới hoặc chịu ảnh hưởng bởi tiến trình khác đang thực hiện trong hệ thống
 - Hợp tác tiến trình nhằm
 - Chia sẻ thông tin
 - Tăng tốc độ tính toán
 - Module hóa
 - Tiện dụng
 - Hợp tác tiến trình đòi hỏi cơ chế cho phép
 - Truyền thông giữa các tiến trình
 - Đồng bộ hóa hoạt động của các tiến trình

1.5. Liên lạc giữa các tiến trình

- **Bài toán người sản xuất (producer) – Người tiêu thụ (consumer)**



- Hệ thống gồm 2 tiến trình
 - **Producer** sản xuất ra các sản phẩm
 - **Consumer** tiêu thụ các sản phẩm được sản xuất ra
- Ứng dụng
 - Chương trình in (producer) sản xuất ra các ký tự được tiêu thụ bởi bộ điều khiển máy in (consumer)
 - Trình dịch (producer) sản xuất ra mã hợp ngữ, trình hợp ngữ (consumer/producer) tiêu thụ mã hợp ngữ rồi sản xuất ra module đối tượng được bộ thực hiện (consumer) tiêu thụ

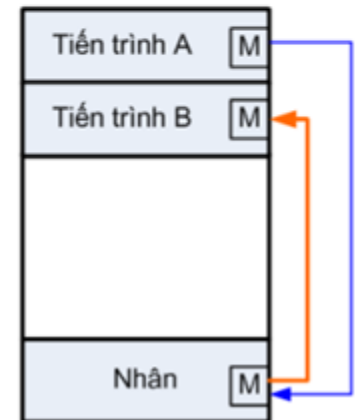
1.5. Liên lạc giữa các tiến trình

- **Bài toán người sản xuất (producer) – Người tiêu thụ (consumer)**
 - **Producer** và **Consumer** hoạt động đồng thời
 - Sử dụng vùng đệm dùng chung (buffer) chứa sản phẩm được điền vào bởi producer và được lấy ra bởi consumer
 - IN** Vị trí trống kế tiếp trong vùng đệm.
 - OUT** Vị trí đầy đầu tiên trong vùng đệm
 - Counter** Số sản phẩm trong vùng đệm
 - Producer và Consumer phải đồng bộ
 - Consumer không cố gắng tiêu thụ một sản phẩm chưa được sản xuất
 - Vùng đệm dung lượng vô hạn
 - Khi Buffer rỗng, Consumer phải đợi
 - Producer không phải đợi khi đặt sản phẩm vào buffer
 - Vùng đệm dung lượng hữu hạn
 - Khi Buffer rỗng, Consumer phải đợi
 - Producer phải đợi nếu vùng đệm đầy

1.5. Liên lạc giữa các tiến trình

• Trao đổi giữa các tiến trình

- Dùng mô hình bộ nhớ phân chia
 - Các tiến trình chia sẻ vùng nhớ chính
 - Mã cài đặt được viết tường minh bởi người lập trình ứng dụng
 - Ví dụ: Bài toán Producer – Consumer
- Dùng mô hình truyền thông liên tiến trình (Interprocess communication)
 - Là cơ chế cho phép các tiến trình truyền thông và đồng bộ các hoạt động
 - Thường được sử dụng trong các hệ phân tán khi các tiến trình truyền thông nằm trên các máy khác nhau (chat)
 - Đảm bảo bởi hệ thống truyền thông điệp



1.5. Liên lạc giữa các tiến trình

Hệ thống truyền thông điệp

- Cho phép các tiến trình trao đổi với nhau không qua sử dụng các biến phân chia
- Yêu cầu 2 thao tác cơ bản
 - Send(msg) Các msg có kích thước cố định hoặc thay đổi
 - Cố định: dễ cài đặt mức hệ thống, nhiệm vụ lập trình khó
 - Thay đổi: cài đặt mức hệ thống phức tạp, lập trình đơn giản
 - Receive(msg)
- Nếu 2 tiến trình P và Q muốn trao đổi, chúng cần
 - Thiết lập một liên kết truyền thông (vật lý / logic) giữa chúng
 - Trao đổi các messages nhờ các thao tác send/receive
- Các vấn đề cài đặt
 - các liên kết được thiết lập như thế nào?
 - Một liên kết có thể dùng cho nhiều hơn 2 tiến trình?
 - Bao nhiêu liên kết có thể tồn tại giữa mọi cặp tiến trình
 - Kích thước thông báo mà liên kết chấp nhận cố định/thay đổi?
 - Liên kết một hay hai chiều?

1.5. Liên lạc giữa các tiến trình

Liên lạc trực tiếp

- Các tiến trình phải gọi tên tiến trình nhận/gửi một cách tường minh
 - `send(P,message)`- gửi một thông báo tới tiến trình P
 - `receive (Q,message)`- Nhận một thông báo tiến trình Q
- Tính chất của liên kết truyền thông
 - Các liên kết được thiết lập tự động
 - Một liên kết gắn chỉ với cặp tiến trình truyền thông
 - Chỉ tồn tại một liên kết giữa cặp tiến trình
 - Liên kết có thể là một chiều, nhưng thường hai chiều

1.5. Liên lạc giữa các tiến trình

Liên lạc gián tiếp

- Các thông điệp được gửi/nhận tới / từ các hòm thư (mailboxes), cổng (ports)
 - Mỗi hòm thư có định danh duy nhất
 - Các tiến trình có thể trao đổi nếu chúng dùng chung hòm thư
- Tính chất các liên kết
 - Các liên kết được thiết lập chỉ khi các tiến trình dùng chung hòm thư
 - Một liên kết có thể được gắn với nhiều tiến trình
 - Mỗi cặp tiến trình có thể dùng chung nhiều liên kết truyền thông
 - Liên kết có thể một hay hai chiều
- Các thao tác
 - Tạo hòm thư
 - Gửi/nhận thông báo qua hòm thư
 - `send(A,msg)`: gửi một msg tới hòm thư A
 - `receive(A,msg)`: Nhận một msg từ hòm thư A
 - Hủy bỏ hòm thư

1.5. Liên lạc giữa các tiến trình

Vấn đề đồng bộ hóa

- Truyền thông điệp có thể phải chờ (blocking), hoặc không chờ đợi (non blocking)

Blocking Truyền thông đồng bộ

Non-Blocking Truyền thông không đồng bộ

- Các thủ tục **send()** và **receive()** có thể bị chờ đợi hoặc không chờ đợi
 - Blocking send Tiến trình gửi thông báo và đợi cho tới khi msg được nhận bởi tiến trình nhận hoặc bởi hàm thư
 - Non Blocking send Tiến trình gửi thông báo và tiếp tục làm việc
 - Blocking receive Tiến trình nhận phải đợi cho tới khi có thông báo
 - Non-blocking receive Tiến trình nhận trả về hoặc một thông báo có giá trị hoặc một giá trị null

1.5. Liên lạc giữa các tiến trình

Vùng đệm

- Các thông điệp trao đổi giữa các tiến trình được lưu trong hàng đợi tạm thời
- Hàng đợi có thể được cài theo
 - Khả năng chứa 0 (Zero capacity): Độ dài hàng đợi là 0
 - Không tồn tại thông điệp trong đường liên kết
 - > Sender phải đợi cho tới khi thông điệp được nhận
 - Khả năng chứa có giới hạn (Bound capacity)
 - Hàng đợi có độ dài n -> chứa nhiều nhất n thông điệp
 - Nếu hàng đợi không đầy, thông điệp sẽ được lưu vào trong vùng đệm và Sender tiếp tục bình thường
 - Nếu hàng đợi đầy, sender phải đợi cho tới khi có chỗ trống
 - Khả năng chứa không giới hạn (Unbound capacity)
 - Sender không bao giờ phải đợi

1.5. Liên lạc giữa các tiến trình

Liên lạc trong client – server

- Socket
- RPC(Remote Procedure Calls)
- RMI(Remote Method Invocation) Cơ chế truyền thông của Java

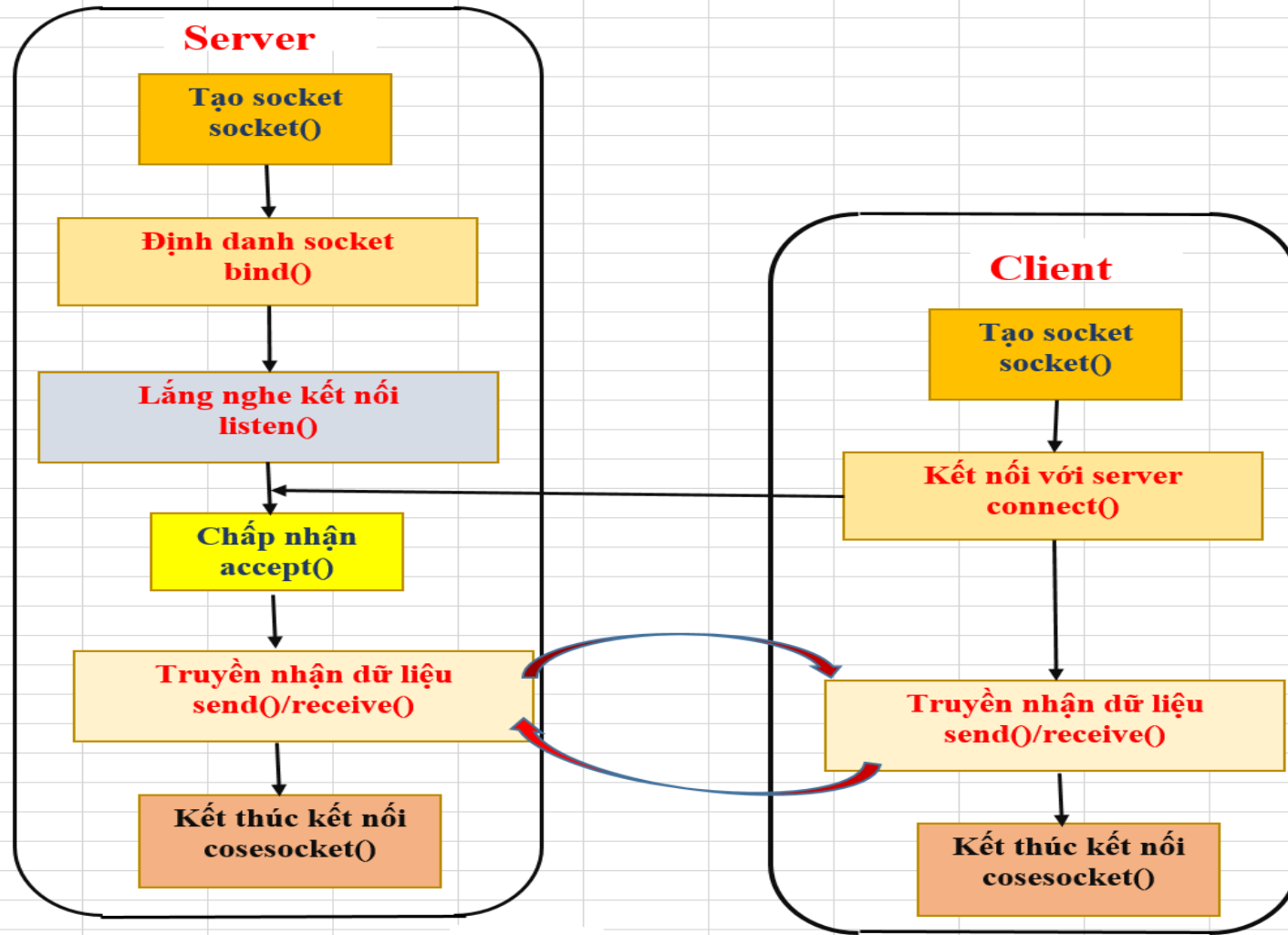
1.5. Liên lạc giữa các tiến trình

Socket

- Được xem như đầu mút cho truyền thông, qua đó các ứng dụng gửi / nhận dữ liệu qua mạng
 - Truyền thông thực hiện giữa các Sockets
- Bao gồm cặp địa chỉ IP và cổng. Ví dụ : 161.25.19.8:1625
 - Địa chỉ IP: Địa chỉ của máy trong mạng
 - Cổng(port): Định danh tiến trình tham gia trao đổi trên máy
- Các loại Sockets
 - Stream Socket: Dựa trên giao thức TCP/IP -> Truyền dữ liệu tin cậy
 - Datagram Socket: Dựa trên giao thức UDP/IP-> Truyền dữ liệu không tin cậy
- Win32 API: Winsock
 - Windows Sockets Application Programming Interface

1.5. Liên lạc giữa các tiến trình

- Thiết lập quá trình trao đổi dữ liệu



1.5. Liên lạc giữa các tiến trình

Một số hàm trong WINAPI

socket()

Tạo socket truyền dữ liệu

bind()

Định danh cho socket vừa tạo(gán cho một cổng)

Listen()

Lắng nghe một kết nối

Accept()

Chấp nhận một kết nối

Connect()

kết nối với server

Send()

Gửi dữ liệu với stream socket

Sendto()

Gửi dữ liệu với datagram socket

Receive()

Nhận dữ liệu với stream socket

Recvfrom()

Nhận dữ liệu với datagram socket

Shutsocket()

Kết thúc một socket đã tồn tại

.....

2. Luồng (Thread)



Process Q

```
While(1){  
  Receive(P,Msg);  
  PrintLine(Msg);  
  ReadLine(Msg);  
  Send(P,Msg);  
}
```

Vấn đề nhận Msg

- Blocking Receive
- Non-blocking Receive

Giải quyết

Thực hiện song song
Receive & Send

Process P

```
While(1){  
  ReadLine(Msg);  
  Send(P,Msg);  
  Receive(P,Msg);  
  PrintLine(Msg);  
}
```

2. Luồng (Thread)

- Là đơn vị sử dụng CPU cơ bản gồm
 - Định danh luồng (ID Thread)
 - Bộ đếm chương trình (Program Counter)
 - Tập các thanh ghi (Registers)
 - Không gian stack
- Chia sẻ cùng các luồng khác trong cùng một tiến trình
 - Đoạn mã lệnh
 - Đoạn dữ liệu (đối tượng toàn cục)
 - Các tài nguyên hệ điều hành khác (file đang mở)
- Các luồng có thể thực hiện cùng đoạn mã với ngữ cảnh (Tập thanh ghi, Bộ đếm chương trình, stack) khác nhau
- Còn được gọi là tiến trình nhẹ (LWP: Lightweight Process)
- Một tiến trình có ít nhất là một luồng

So sánh luồng với tiến trình

Tiêu chí	Tiến trình	Luồng
Định nghĩa	Tiến trình là một chương trình đang được thực thi, tức là một chương trình đang hoạt động.	Luồng là một tiến trình gọn nhẹ có thể được quản lý độc lập bởi một bộ lập lịch.
Thời gian chuyển đổi ngữ cảnh	Các tiến trình đòi hỏi nhiều thời gian hơn để chuyển đổi ngữ cảnh vì chúng nặng hơn.	Các luồng yêu cầu ít thời gian hơn để chuyển đổi ngữ cảnh vì chúng nhẹ hơn tiến trình.
Chia sẻ bộ nhớ	Các tiến trình hoàn toàn độc lập và không chia sẻ bộ nhớ.	Một luồng có thể chia sẻ một số bộ nhớ với các luồng ngang hàng của nó.
Giao tiếp	Giao tiếp giữa các tiến trình đòi hỏi nhiều thời gian hơn giữa các luồng.	Giao tiếp giữa các luồng yêu cầu ít thời gian hơn giữa các quá trình.
Khi bị chặn	Nếu một tiến trình bị chặn, các tiến trình còn lại có thể tiếp tục thực thi.	Nếu một luồng cấp người dùng bị chặn, tất cả các luồng ngang hàng của nó cũng bị chặn.
Tiêu thụ tài nguyên	Tiến trình yêu cầu nhiều tài nguyên hơn luồng.	Luồng thường cần ít tài nguyên hơn các tiến trình.
Sự phụ thuộc	Các tiến trình riêng lẻ độc lập với nhau.	Luồng là các phần của một tiến trình và do đó chúng mang tính phụ thuộc.
Chia sẻ dữ liệu và code	Các tiến trình có dữ liệu và code segment độc lập.	Một luồng chia sẻ data segment, code segment, file, v.v... với các luồng ngang hàng
Cách hệ điều hành nhìn nhận	Tất cả các tiến trình khác nhau được hệ điều hành xử lý riêng biệt.	Tất cả các luồng ngang hàng cấp người dùng được hệ điều hành coi như một tác vụ duy nhất.
Thời gian tạo	Các tiến trình đòi hỏi nhiều thời gian hơn để tạo.	Các luồng yêu cầu ít thời gian hơn để tạo.
Thời gian chấm dứt	Các tiến trình yêu cầu nhiều thời gian hơn để chấm dứt.	Luồng yêu cầu ít thời gian hơn để kết thúc.

Lợi ích của lập trình đa luồng

- Tăng tính đáp ứng với người dùng
 - Cho phép chương trình vẫn thực hiện ngay khi một phần đang chờ đợi (block) hoặc đang thực hiện tính toán tăng cường
 - Ví dụ trình duyệt Web (Web browser) đa luồng
 - Một luồng tương tác với người dùng
 - Một luồng thực hiện nhiệm vụ tải dữ liệu
- Chia sẻ tài nguyên
 - Các luồng chia sẻ bộ nhớ và tài nguyên của tiến trình chứa nó
 - Tốt cho các thuật toán song song (Sử dụng chung các CTDL)
 - Trao đổi giữa các luồng thông qua bộ nhớ phân chia
 - Cho phép một ứng dụng chứa nhiều luồng hoạt động trong cùng không gian địa chỉ
- Tính kinh tế
 - Các thao tác khởi tạo, hủy bỏ và luân chuyển luồng ít tốn kém
 - Minh họa được tính song song trên bộ đơn VXL do thời gian luân chuyển CPU nhanh (Thực tế chỉ một luồng thực hiện)
- Sử dụng kiến trúc nhiều vi xử lý
 - Các luồng chạy song song thực sự trên các bộ VXL khác nhau

2. Luồng (Thread)

- Có 2 loại luồng cơ bản:
 - Luồng mức người dùng (User – Level Threads)
 - Luồng mức hệ thống (Kernel - Level Threads)

Luồng mức người dùng (User – Level Threads)

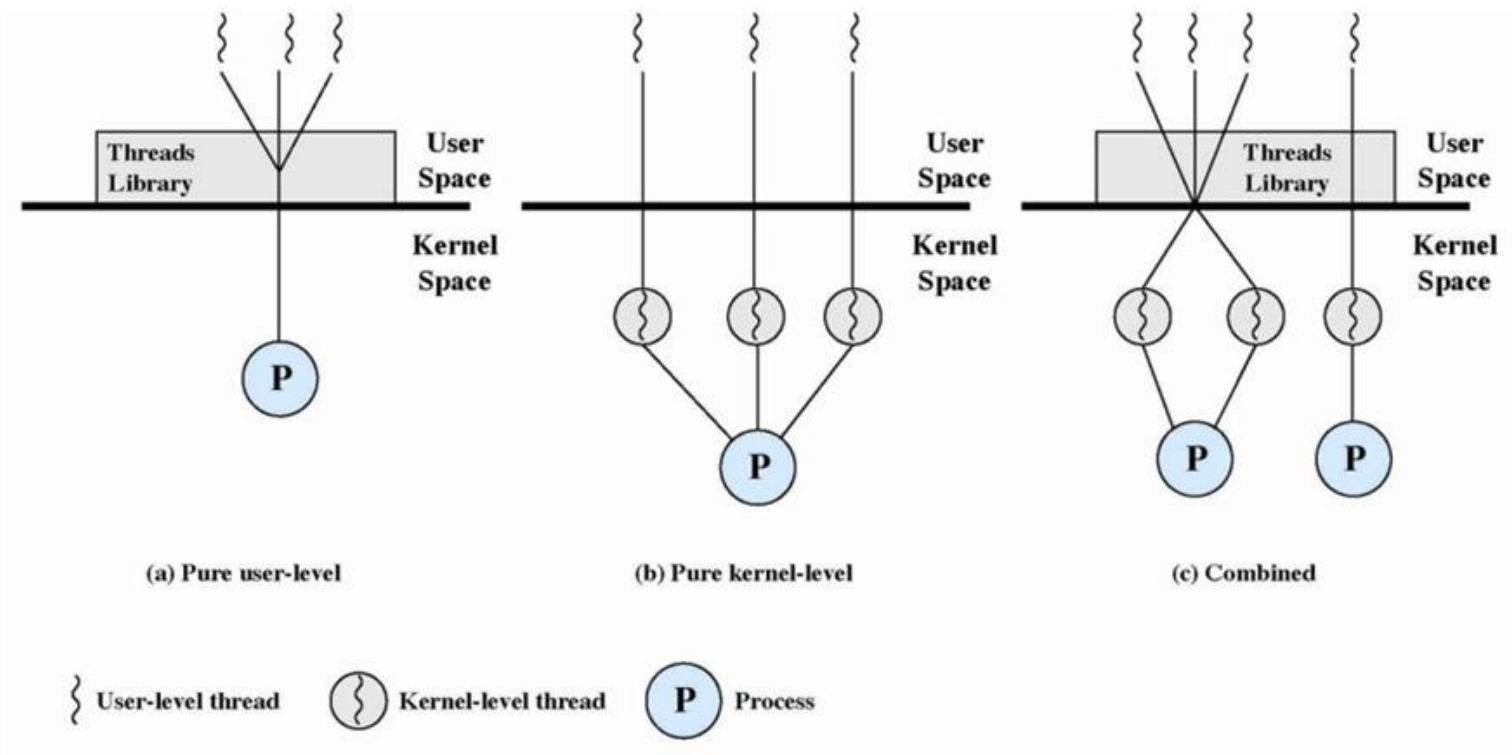
- Quản lý các luồng được thực hiện bởi chương trình ứng dụng
- Nhân hệ thống không biết gì về sự tồn tại luồng
 - Điều phối tiến trình như một đơn vị duy nhất
 - Gán cho mỗi tiến trình một trạng thái duy nhất
 - Sẵn sàng, chờ đợi, thực hiện,..
- Chương trình ứng dụng được lập trình theo mô hình đa luồng bởi sử dụng thư viện luồng
 - Thư viện hỗ trợ tạo, hủy bỏ, truyền thông điệp giữa các luồng điều phối, lưu trữ, khôi phục trạng thái (context) luồng ,..
- Ưu điểm
 - Nhanh chóng trong tạo và quản lý luồng
- Nhược điểm
 - Khi một luồng rơi vào trạng thái chờ đợi, tất cả các luồng trong cùng tiến trình bị chờ đợi theo =>Không tận dụng được ưu điểm của mô hình lập trình đa luồng

Luồng mức hệ thống (Kernel - Level Threads)

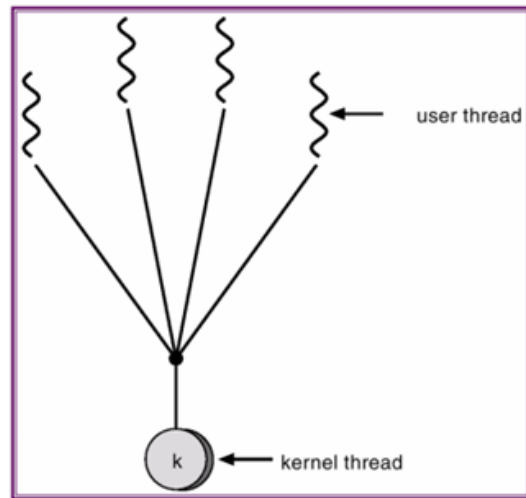
- Nhân duy trì thông tin về tiến trình và các luồng
- Quản lý luồng được thực hiện bởi nhân
 - Không tồn tại các mã quản lý luồng trong ứng dụng
 - Điều phối luồng được thực hiện bởi nhân, dựa trên các luồng
- Nhược điểm:
 - Chậm trong tạo và quản lý luồng
- Ưu điểm:
 - Một luồng chờ đợi vào ra, không ảnh hưởng tới luồng khác
 - Trong môi trường đa VXL, nhân có thể điều phối các luồng cho các VXL khác nhau
- Hệ điều hành: Windows NT/2000/XP, Linux, OS/2,...

2.2 Mô hình đa luồng

Nhiều hệ thống hỗ trợ cả luồng mức người dùng và luồng mức hệ thống
-> Nhiều mô hình đa luồng khác nhau

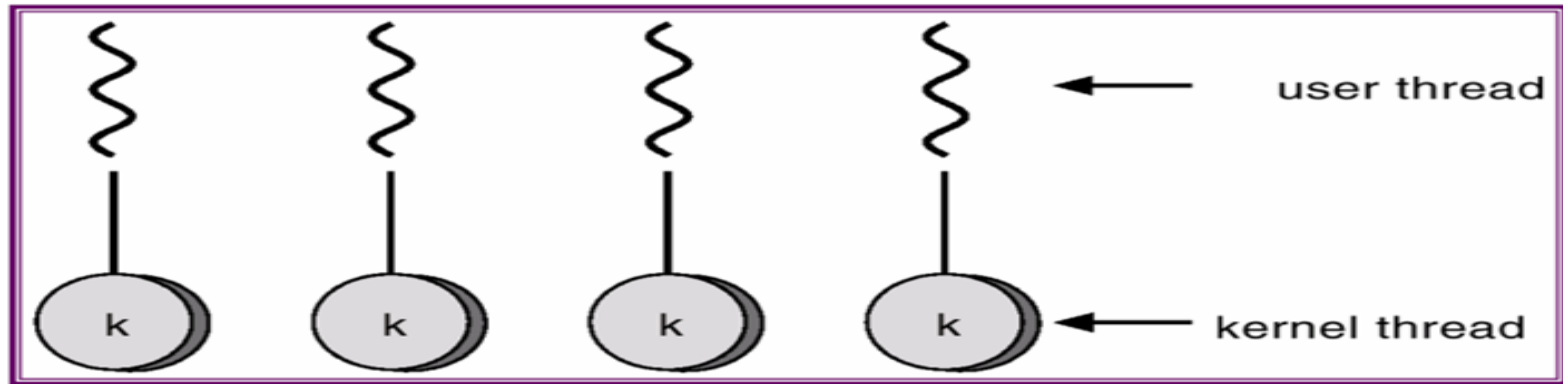


Mô hình nhiều – một



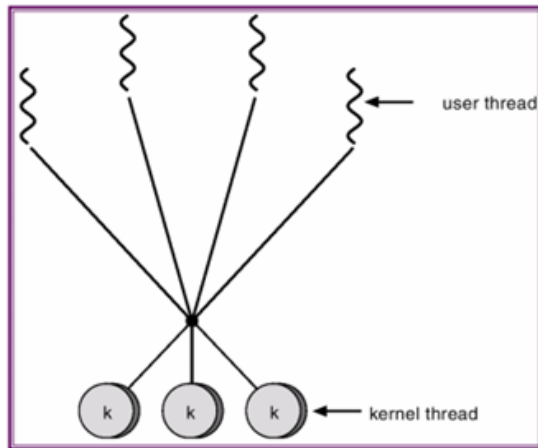
- Ánh xạ nhiều luồng mức người dùng tới một luồng mức hệ thống
- Quản lý luồng được thực hiện trong không gian người dùng
 - Hiệu quả
 - Cho phép tạo nhiều luồng tùy ý
 - Toàn bộ tiến trình sẽ bị khóa nếu một luồng bị khóa
- Không thể chạy song song trên các máy nhiều vi xử lý (chỉ một luồng có thể truy nhập nhân tại một thời điểm)
- Dùng trong hệ điều hành không hỗ trợ hệ thống

Mô hình một – một



- Ánh xạ mỗi luồng mức người dùng tới một luồng hệ thống
 - Cho phép thực hiện luồng khác khi một luồng bị chờ đợi
 - Cho phép chạy song song đa luồng trên máy nhiều vi xử lý
- Tạo luồng mức người dùng đòi hỏi tạo một luồng mức hệ thống tương ứng
 - Ảnh hưởng tới hiệu năng của ứng dụng
 - Chi phí cao -> Giới hạn số luồng được hệ thống hỗ trợ
- Được sử dụng trong Window NT/2000/XP

Mô hình nhiều – nhiều



- Nhiều luồng mức người dùng ánh xạ tới một số nhỏ luồng mức hệ thống
- Số lượng luồng nhân có thể được xác định tho máy hoặc theo ứng dụng
 - VD: Được cấp nhiều luồng nhân hơn trên hệ thống nhiều VXL
- Có được ưu điểm của 2 mô hình trên
 - Cho phép tạo nhiều luồng mức ứng dụng theo yêu cầu
 - Các luồng nhân tương ứng có thể chạy song song trên hệ nhiều VXL
 - Một luồng bị khóa, nhân có thể cho phép luồng khác thực hiện
- Ví dụ: UNIX

2.3. Cài đặt luồng

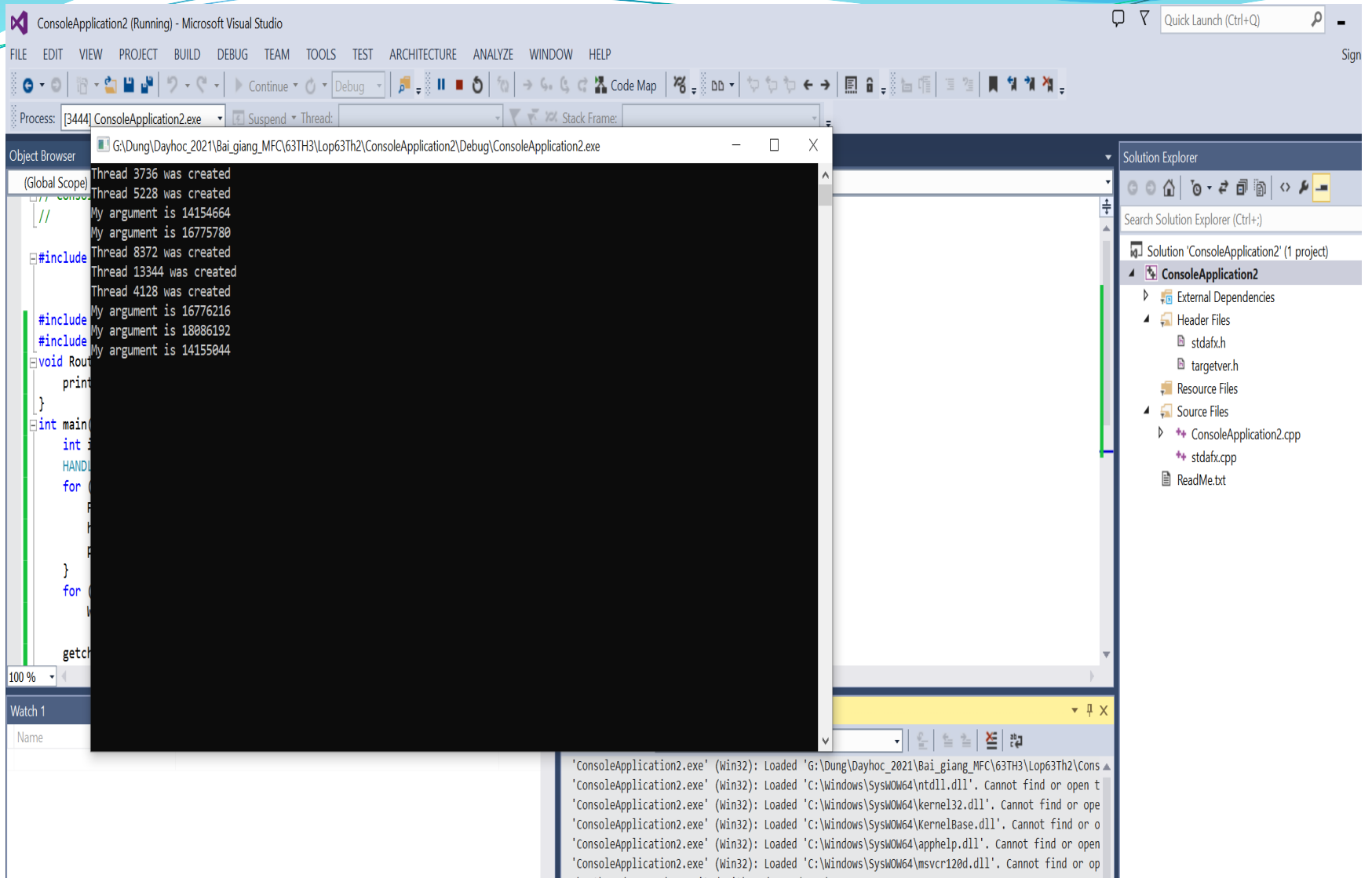
- Cài đặt trong hệ điều hành windows

Một số hàm với luồng trong WIN32 API

- HANDLE CreateThread(...);
 - LPSECURITY_ATTRIBUTES lpThreadAttributes,
=>Trở tới cấu trúc an ninh: thẻ trả về có thể được kế thừa?
 - DWORD dwStackSize,
=>Kích thước ban đầu của stack cho luồng mới
 - LPTHREAD_START_ROUTINE lpStartAddress,
=>Trở tới hàm được thực hiện bởi luồng mới
 - LPVOID lpParameter,
=>Trở tới các biến được gửi tới luồng mới (*tham số của hàm*)
 - DWORD dwCreationFlags,
=>Phương pháp tạo luồng
 - CREATE_SUSPENDED : Luồng ở trạng thái tạm ngừng
 - 0: Luồng được thực hiện ngay lập tức
 - LPDWORD lpThreadId
=>Biến ghi nhận định danh luồng mới
- Kết quả trả về: Thẻ của luồng mới hoặc giá trị NULL nếu không tạo được luồng mới

```
#include <windows.h>
#include <stdio.h>
void Routine(int *n) {
    printf("My argument is %d\n", &n);
}
int main() {
    int i, P[5]; DWORD Id;
    HANDLE hHandles[5];
    for (i = 0; i < 5; i++) {
        P[i] = i;
        hHandles[i] = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)Routine, &P[i], 0, &Id);
        printf("Thread %d was created\n", Id);
    }
    for (i = 0; i < 5; i++)
        WaitForSingleObject(hHandles[i], INFINITE);

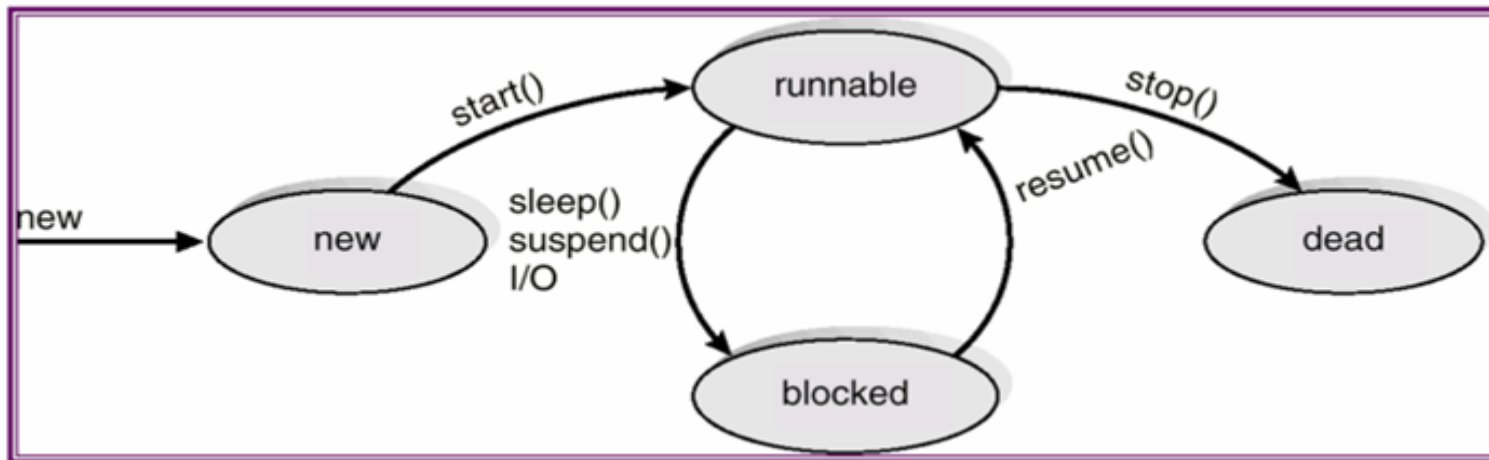
    getchar();
    return 0;
}
```



2.3. Cài đặt luồng

- Cài đặt với Java Threads

- Được cài đặt bởi
 - Mở rộng lớp Thread (*Thread class*)
 - Cài đặt giao diện có thể thực thi được (*Runnable interface*)
- Được quản lý bởi máy ảo Java (Java Virtual Machine)
- Các trạng thái có thể



- Tồn tại một phương thức **run()**, sẽ được thực hiện trên JVM
- Luồng được thực hiện bởi gọi phương thức **start()**
 - Cung cấp vùng nhớ và khởi tạo luồng mới trong máy ảo Java
 - Gọi tới phương thức **run()**

MultithreadingDemo1.java

```
18         }
19     }
20     catch (InterruptedException e)
21     {
22         System.out.println("my thread interrupted");
23     }
24     System.out.println("mythread run is over" );
25 }
26 }
27 class MultithreadingDemo1
28 {
29     public static void main(String args[])
30     {
31         Count cnt = new Count();
32         try
33         {
34             while(cnt.mythread.isAlive())
35             {
36                 System.out.println("Main thread will be alive
```

TERMINAL

DEBUG CONSOLE

PROBLEMS

OUTPUT

```
Main thread will be alive till the child thread is live
Printing the count 4
Main thread will be alive till the child thread is live
Printing the count 5
Main thread will be alive till the child thread is live
Printing the count 6
Printing the count 7
Main thread will be alive till the child thread is live
Printing the count 8
Main thread will be alive till the child thread is live
Printing the count 9
mythread run is over
Main thread run is over
```

2.4. Vấn đề đa luồng

```
#include <windows.h>
#include <stdio.h>
int x = 0, y = 1;
void T1() {
    while (1) { x = y + 1; printf("%4d", x); }
}
void T2() {
    while (1) { y = 2; y = y * 2; }
}
int main() {
    HANDLE h1, h2; DWORD Id;
    h1 = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)T1, NULL, 0, &Id);
    h2 = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)T2, NULL, 0, &Id);
    WaitForSingleObject(h1, INFINITE);
    WaitForSingleObject(h2, INFINITE);
    getchar();
    return 0;
}
```

Trường hợp 1

Shared int $y=1$

Thread T1	Thread T2
$x \leftarrow y+1$	$y \leftarrow 2$
	$y \leftarrow y*2$

$x=?$

Thread A

Thread B

$x \leftarrow y+1$

t_1

$y \leftarrow 2$

t_a

t_2

$y \leftarrow y*2$

$x=3$



Trường hợp 2

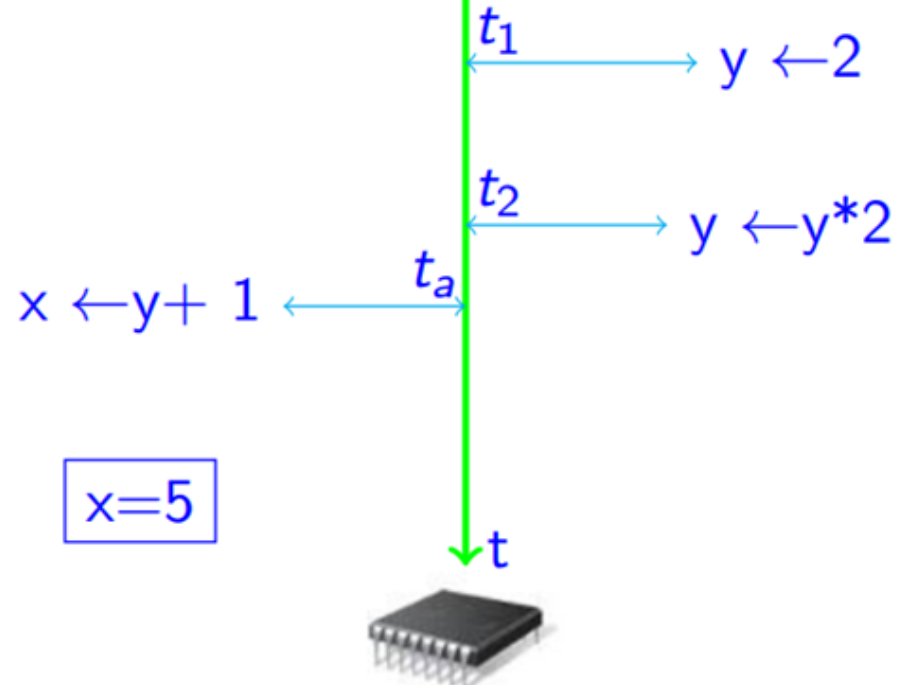
Shared int y=1

Thread T1	Thread T2
$x \leftarrow y+1$	$y \leftarrow 2$
	$y \leftarrow y*2$

x=?

Thread A

Thread B



Trường hợp 3

Shared int y=1

Thread T1	Thread T2
$x \leftarrow y+1$	$y \leftarrow 2$
	$y \leftarrow y*2$

x=?

Kết quả thực hiện các luồng song song phụ thuộc trật tự truy nhập biến dùng chung giữa chúng

Thread A

Thread B

